

In this notebook, I have tried my best to the UNet architecture with the help of illustrations and in-depth descriptions along with its implementation from scratch. Feedback would be greatly appreciated :)

Learnings Objectives of this Notebook

- What is UNet Architecture?
- Working and Breakdown of UNet Architecture.
- Practicle Implementation of UNet.
- Applications of UNet Architecture.

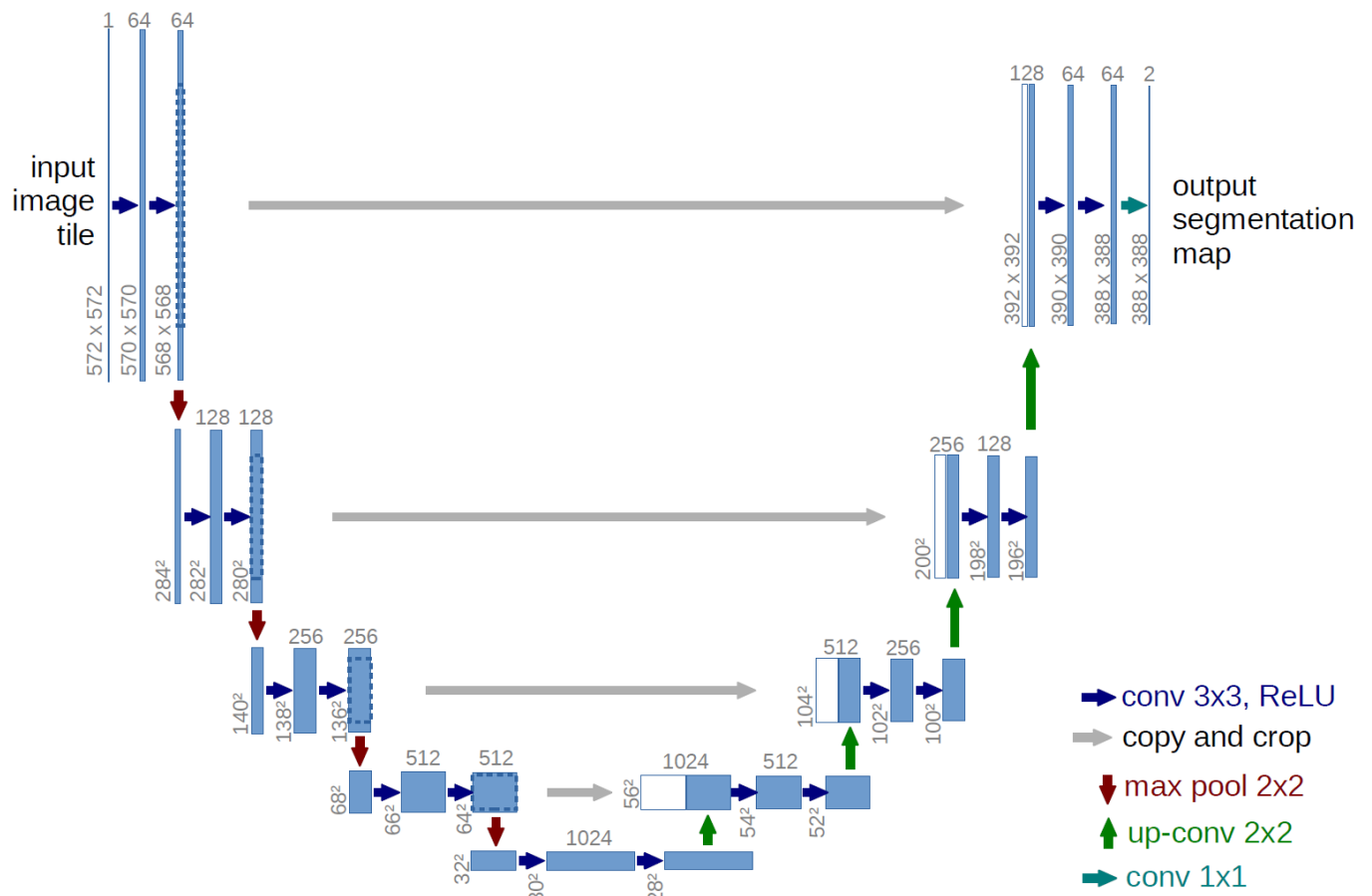
This notebook is a one stop destination to learn everything about **UNet Architecture**.

What is UNet Architecture?

The U-Net is a convolutional neural network initially developed for biomedical image segmentation at the University of Freiburg, Germany. It is an extension of the fully convolutional network proposed by Long, Shelhamer, and Darrell.

The **U-Net architecture** enhances the segmentation accuracy by incorporating upsampling operators instead of pooling operations, which increases the output resolution. Additionally, the network employs a **symmetric u-shaped structure**. This design choice enables the network to capture and propagate context information effectively. By utilizing a large number of feature channels in the upsampling part, the network can extract and utilize rich contextual information, leading to more accurate segmentations.

Unlike traditional networks that rely on fully connected layers, the U-Net does not use them and instead extrapolates missing context in the border region by **mirroring the input image**. This approach enables the network to handle large images efficiently.



Breakdown of UNet Architecture

The U-Net architecture consists of an encoder-decoder structure with skip connections, enabling it to capture both high-level and low-level features.

Here's a breakdown of the U-Net architecture:

1. **Encoder:** The encoder is responsible for capturing high-level features from the input image.

- It starts with a series of convolutional layers with a **ReLU activation** function, followed by **padding** to maintain spatial dimensions.
- **Convolutional layers** apply a set of filters to extract features.
- After each convolutional layer, another convolutional layer with the same number of filters is applied to capture more complex features.
- **MaxPooling** is performed to downsample the feature maps and reduce spatial dimensions.

2. **Bridge:** The bridge connects the encoder and decoder through skip connections.

- It consists of additional convolutional layers with **ReLU activation**.
- The bridge helps in preserving spatial information by concatenating the feature maps from the encoder to the corresponding decoder layers.

3. **Decoder:** The decoder generates the final segmentation map using the concatenated feature maps from the bridge.

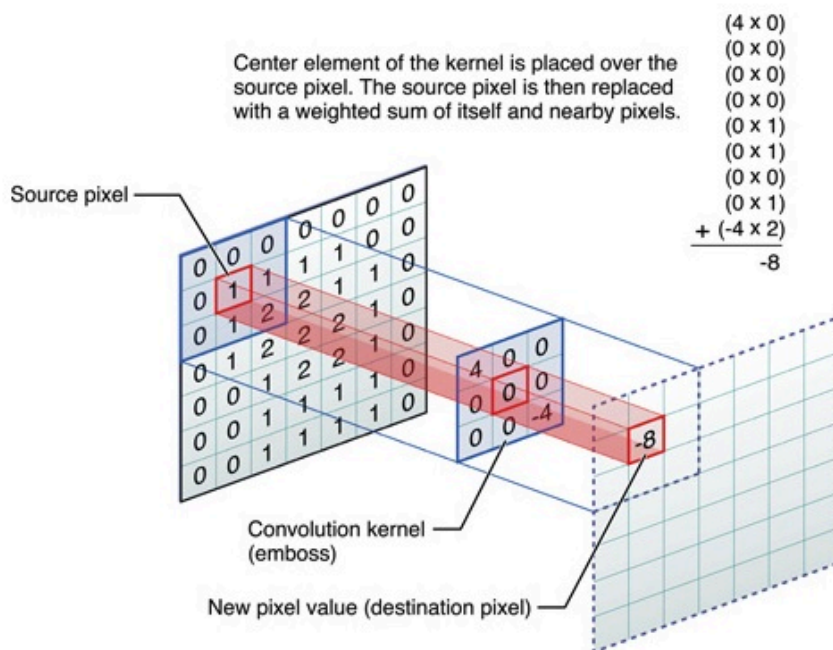
- It starts with **upsampling** to increase the spatial dimensions of the feature maps.
- **Convolutional layers** with **ReLU activation** are applied to refine the feature maps.
- The **upsampled feature maps** are concatenated with the corresponding feature maps from the encoder.
- More convolutional layers are applied to further refine the features.
- The decoder ends with a **convolutional layer with a sigmoid activation** function to produce the final segmentation map.

The U-Net architecture leverages skip connections to combine both high-level and low-level features, allowing it to capture fine-grained details while maintaining contextual information. This makes it effective for tasks such as image segmentation, where precise delineation of object boundaries is important.

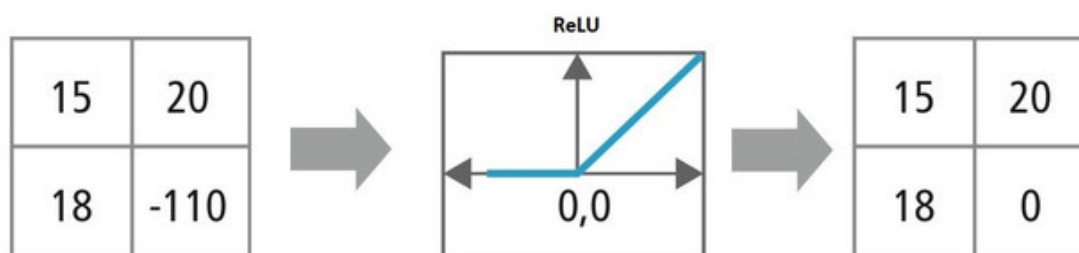
Let's go through the basics of convolution & deconvolution

Understanding Convolution Operation

Convolution operation is an essential technique used in convolutional neural networks (CNNs) for image classification. It involves using a feature detector/kernel/filter of a certain size (e.g., 3x3, 5x5, or 7x7) to detect specific features in an image, such as edges or shapes. The feature detector convolves or slides over the input image, performing element-wise multiplication of the corresponding pixel values. This results in a feature map or convolved feature that highlights the detected feature in the image. We use multiple feature detectors to create multiple feature maps, which are important for accurate classification. The stride is the number of steps that the feature detector takes while navigating over the input image. The size of the input image is reduced during convolution, which is useful for faster processing, but it also results in some loss of information.



After applying the convolutional layer, ReLU activation function is used to introduce non-linearity in the model. This helps in breaking the linearity of the image data and improving the model's ability to capture non-linear features in the image (as most of the data is non-linear in nature, we need to introduce back non-linearity)



Understanding Deconvolution Operation

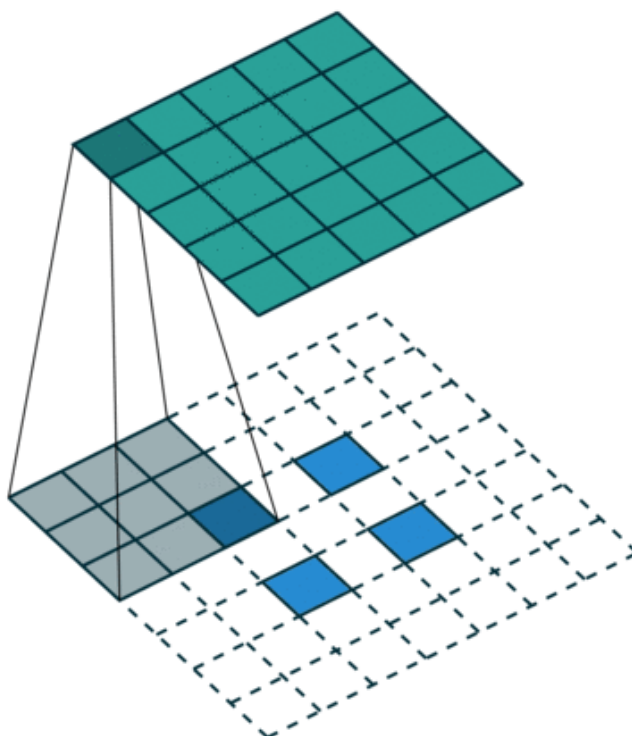
The deconvolution operation is used to increase the spatial dimensions of feature maps. It is the inverse operation of convolution and is often employed in the decoder part of architectures like U-Net for upsampling.

Here's a breakdown of the deconvolution operation:

1. **Upsampling:** The deconvolution operation starts with **upsampling** the input feature map to increase its spatial dimensions. This is typically done by inserting empty rows and columns between the existing elements.
2. **Convolution:** After upsampling, a **convolutional layer** is applied to the upsampled feature map. The convolutional layer applies a set of learnable filters to the feature map, enabling it to learn patterns and extract features.
3. **Stride:** The deconvolution operation involves using a **stride value greater than 1** during the convolution step. The stride determines the step size used when sliding the filters over the input. A stride greater than 1 increases the spatial dimensions of the output feature map.
4. **Padding:** To ensure that the output feature map has the desired spatial dimensions, **padding** can be applied before the convolution step. Padding inserts additional rows and columns of zeros around the input feature map, preserving its size during the convolution operation.
5. **Activation:** Finally, an **activation function** is typically applied to introduce non-linearity and make the deconvolution operation capable of modeling complex relationships between features.

The deconvolution operation is commonly used in tasks such as image upsampling, where low-resolution images need to be enlarged to match higher resolutions. It is also used in image generation tasks, such as in Generative Adversarial Networks (GANs), to produce realistic and high-resolution images.

To visualize deconvolution operation, you can refer to the above gif



implementing UNET From Scratch

In the original paper, the UNET is described as follows:

2

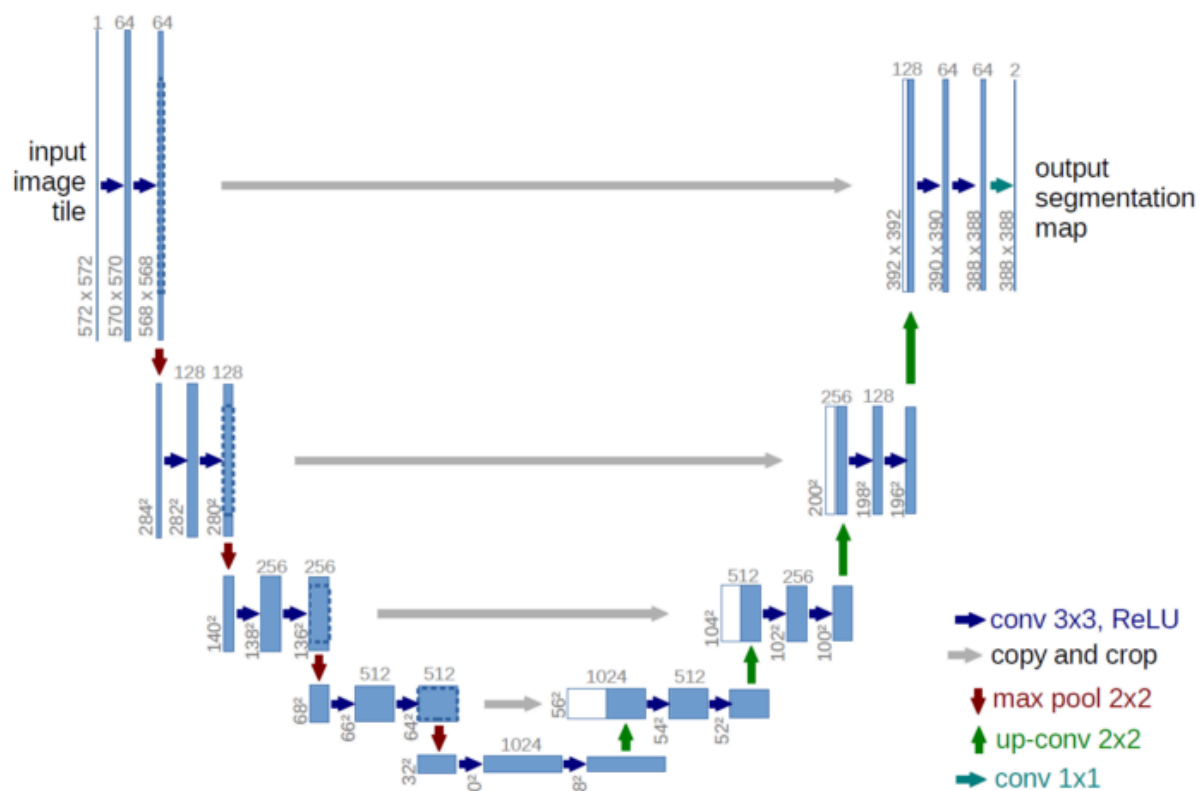


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

From the above figure, we can interpret-

- The U-Net architecture consists of two consecutive convolutional layers in each block
- The left-hand side of the U-Net architecture corresponds to the **contraction path (Encoder)**. This path involves applying regular convolutions and max pooling layers.
- In the Encoder, the input image gradually reduces in size while the depth (number of channels) increases. For example, the image may go from 572x572x3 to 284x284x128
- The Encoder learns the "WHAT" information in the image, but it loses the "WHERE" information, which refers to precise localization.
- The right-hand side of the U-Net architecture represents the **expansion path (Decoder)**. This path applies transposed convolutions along with regular convolutions.
- In the Decoder, the size of the image gradually increases, while the depth decreases. For instance, the image may go from 8x8x256 to 128x128x1.
- The Decoder recovers the "WHERE" information by gradually applying up-sampling to obtain precise localization.
- To achieve more accurate localization, **skip connections** are employed. These connections involve concatenating the output of transposed convolutional layers with the corresponding feature maps from the Encoder:
 - $u_6 = u_6 + c_4$
 - $u_7 = u_7 + c_3$
 - $u_8 = u_8 + c_2$
 - $u_9 = u_9 + c_1$

After concatenation, two consecutive regular convolutions are applied to refine the output.

- The U-Net architecture's symmetric U-shape, along with skip connections, contributes to its name: **UNET**.

Now, we implement the UNET architecture **as per the paper** (done in pytorch)


```
import torch
import torch.nn as nn

class conv_block(nn.Module):
    def __init__(self, inp, out, paddi=0):
        super().__init__()

        self.conv1 = nn.Conv2d(inp, out, kernel_size=3, padding=paddi)
        self.conv2 = nn.Conv2d(out, out, kernel_size=3, padding=paddi)

        self.relu = nn.ReLU()

    def forward(self, inputs):
        x = self.conv1(inputs)

        x = self.relu(x)

        x = self.conv2(x)

        x = self.relu(x)

        return x

class downsample_block(nn.Module):
    def __init__(self, inp, out):
        super().__init__()

        self.conv = conv_block(inp, out)
        self.pool = nn.MaxPool2d((2, 2))

    def forward(self, inputs):
        x = self.conv(inputs)
        p = self.pool(x)

        return x, p

class upsample_block(nn.Module):
    def __init__(self, inp, out):
        super().__init__()

        self.up = nn.ConvTranspose2d(inp, out, kernel_size=2, stride=2)
        self.conv = conv_block(out+out, out)

    def forward(self, inputs, skip):
        x = self.up(inputs)
```

```
x = torch.cat([x, skip], axis=1)
x = self.conv(x)

return x

class build_unet(nn.Module):
    def __init__(self):
        super().__init__()

        self.d1 = downsample_block(1, 64)
        self.d2 = downsample_block(64, 128)
        self.d3 = downsample_block(128, 256)
        self.d4 = downsample_block(256, 512)

        self.b = conv_block(512, 1024, 1)

        self.u1 = upsample_block(1024, 512)
        self.u2 = upsample_block(512, 256)
        self.u3 = upsample_block(256, 128)
        self.u4 = upsample_block(128, 64)

        self.outputs = nn.Conv2d(64, 1, kernel_size=1, padding=0)

    def forward(self, inputs):

        s1, p1 = self.d1(inputs)
        s2, p2 = self.d2(p1)
        s3, p3 = self.d3(p2)
        s4, p4 = self.d4(p3)

        b = self.b(p4)

        u1 = self.u1(b, s4)
        u2 = self.u2(u1, s3)
        u3 = self.u3(u2, s2)
        u4 = self.u4(u3, s1)

        outputs = self.outputs(u4)

        return outputs
```

Applications of UNet Architecture

- **Medical Image Segmentation:** UNet is extensively used for segmenting organs, tumors, and structures in medical images like MRI scans and CT scans, providing precise and detailed segmentation results.
- **Object Detection and Instance Segmentation:** UNet can be adapted to handle object detection and instance segmentation tasks in computer vision, enabling accurate detection and localization of objects in images and videos.
- **Satellite Image Analysis:** UNet finds applications in satellite image analysis, such as land cover classification, building detection, and road extraction, contributing to remote sensing and geospatial analysis.
- **Autonomous Driving:** UNet-based models are employed in autonomous driving systems for tasks like semantic segmentation, enabling vehicles to perceive and understand the environment accurately, enhancing safety and decision-making.
- **Robotics:** UNet is utilized in robotics for tasks like scene understanding, object recognition, and manipulation, allowing robots to perceive and interact with their surroundings effectively.
- **Biomedical Research:** UNet plays a vital role in biomedical research, assisting in cell and tissue segmentation, disease detection, and analysis of microscopic images.
- **Natural Language Processing:** UNet has even found applications in natural language processing, specifically in text segmentation tasks, such as segmenting paragraphs, sentences, or words from textual data.
- **Image Restoration and Denoising:** UNet can be employed for image restoration tasks, such as denoising, inpainting, and super-resolution, by effectively capturing and restoring missing or corrupted information in images.